

Sprawozdanie z pracowni specjalistycznej

Systemy wbudowane

Tytuł: *Liczniki*

Wykonujący: *Monika Słowikowska, Filip Rutkowski*

Numer grupy PS: 9

Numer zespołu: 6

Numer pracy laboratoryjnej: 6

Studia dzienne

Kierunek: Informatyka

Semestr: VI

Prowadzący ćwiczenie: prof. dr hab. inż. Salauyou Valery

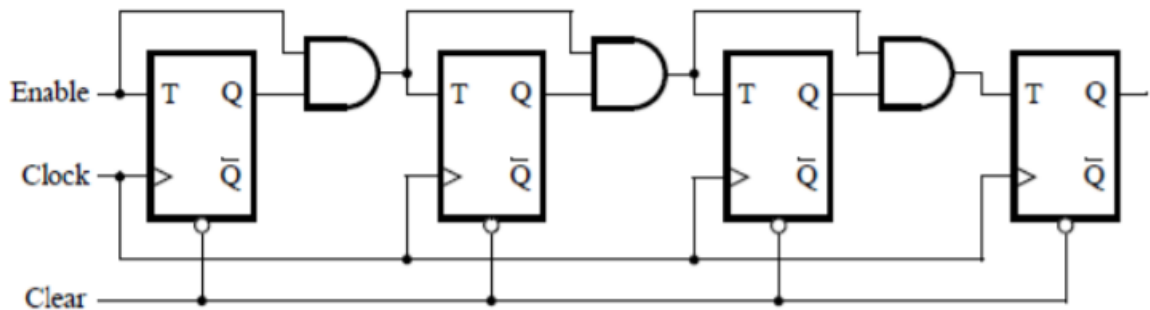
1. Zadanie 1

Utwórz projekt `counter_T_4_bits` dla 4-bitowego licznika na przerzutnikach T z następującym przyporządkowaniem pinów:

Wyprowadzenia	Piny
clk	KEY[0]
areset	SW[0]
enable	SW[1]
Q	HEX0

Gdzie: clk – sygnał zegara;
areset – asynchroniczne ustawianie;
enable – zezwolenie na zliczanie;
Q – rezultat.

Schemat układu:



Opis działania przerzutnika T:

```
always @(posedge clk, negedge aclr)
  if (!aclr) Q <= 1'b0;
  else if (T) Q <= ~Q;
  else Q <= Q;
```

Kod w języku Verilog:

```
module counter_T_4_bits(  
    input [0:0] KEY,  
    input [0:1] SW,  
    output [0:6] HEX0);  
  
    wire clk, enable, reset;  
    assign clk = KEY[0];  
    assign enable = SW[1];  
    assign reset = ~SW[0];  
  
    wire Q1, Q2, Q3, Q4;  
    wire T2in, T3in, T4in;  
  
    assign T2in = enable & Q1;  
    assign T3in = T2in & Q2;  
    assign T4in = T3in & Q3;  
  
    T_flipflop ex1(enable, clk, Q1, reset);  
    T_flipflop ex2(T2in, clk, Q2, reset);  
    T_flipflop ex3(T3in, clk, Q3, reset);  
    T_flipflop ex4(T4in, clk, Q4, reset);  
  
    wire [0:3] Q;  
    assign Q[0] = Q4;  
    assign Q[1] = Q3;  
    assign Q[2] = Q2;  
    assign Q[3] = Q1;  
  
    hex_decoder h(Q, HEX0);  
  
endmodule
```

Główny moduł.

```
module T_flipflop (T, clk, Q, aclr);  
    input T, clk, aclr;  
    output reg Q;  
  
    always @ (posedge clk, negedge aclr)  
        if (!aclr) Q <= 1'b0;  
        else if (T) Q <= ~Q;  
        else Q <= Q;  
endmodule
```

Przerzutnik typu T.

```

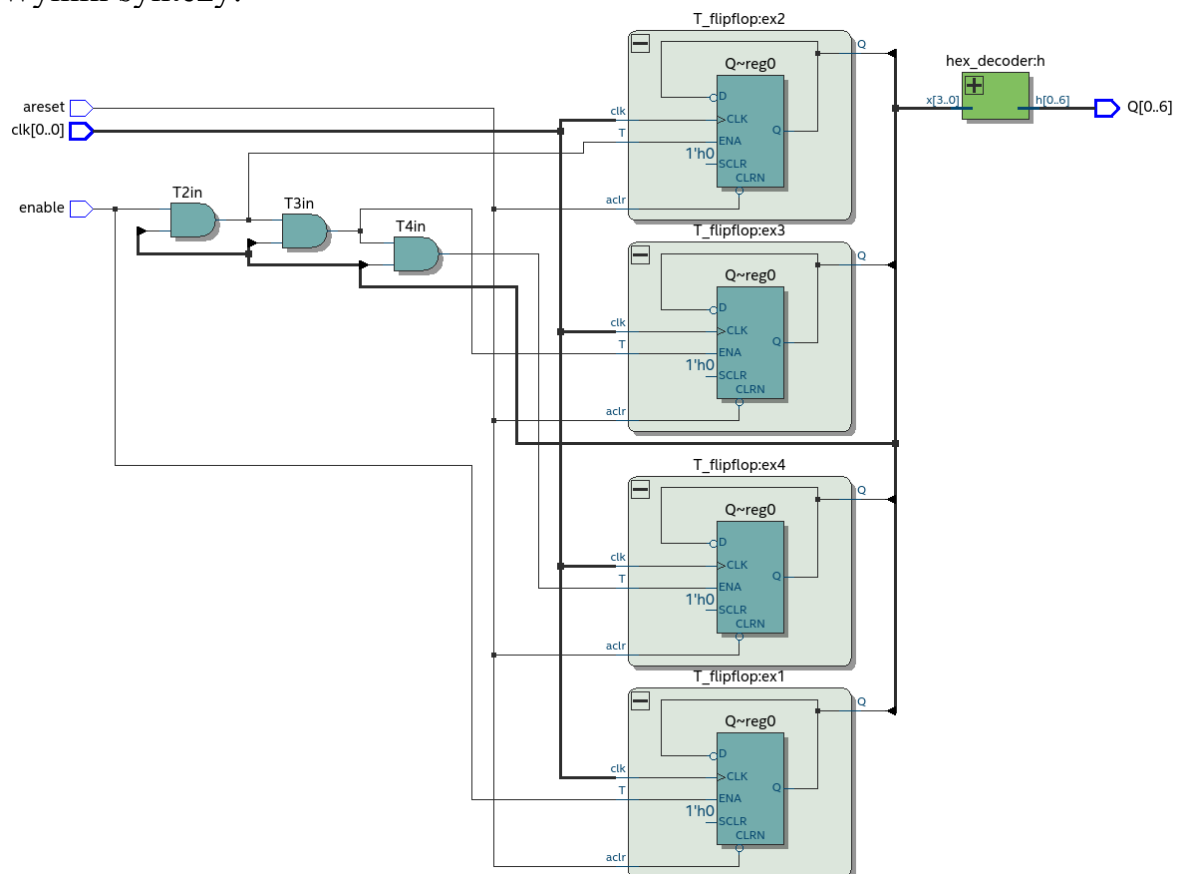
module hex_decoder (
    input [3:0] x,
    output reg [0:6] h);

    always @(*)
        case(x)
            4'b0000: h = 7'b0000001; // 0
            4'b0001: h = 7'b1001111; // 1
            4'b0010: h = 7'b0010010; // 2
            4'b0011: h = 7'b0000110; // 3
            4'b0100: h = 7'b1001100; // 4
            4'b0101: h = 7'b0100100; // 5
            4'b0110: h = 7'b0100000; // 6
            4'b0111: h = 7'b0001111; // 7
            4'b1000: h = 7'b0000000; // 8
            4'b1001: h = 7'b0000100; // 9
            4'b1010: h = 7'b0001000; // A
            4'b1011: h = 7'b1100000; // b
            4'b1100: h = 7'b0110001; // C
            4'b1101: h = 7'b1000010; // d
            4'b1110: h = 7'b0110000; // E
            4'b1111: h = 7'b0111000; // F
        endcase
endmodule

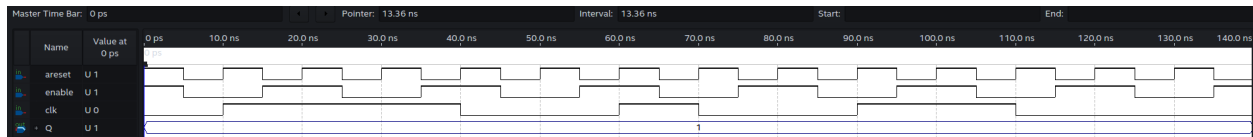
```

Dekoder szesnastkowy.

Wyniki syntezy:



Wyniki symulacji:



2. Zadanie 2

Utwórz projekt counter_16_bits 16-bitowego licznika, wykorzystując jego behawioralny / algorytmiczny opis za pomocą operatora: $Q \leq Q + 1$. Wyświetl wynik na wyświetlaczach HEX3-0.

Kod w języku Verilog:

```
module counter_16_bits(  
    input [1:0] KEY,  
    output [0:6] HEX0, HEX1, HEX2, HEX3);  
  
    wire clk, aclr;  
    assign clk = ~KEY[1];  
    assign aclr = KEY[0];  
  
    reg [15:0] Q;  
  
    hex_decoder hex_decoder4 (Q[15:12], HEX3);  
    hex_decoder hex_decoder3 (Q[11:8], HEX2);  
    hex_decoder hex_decoder2 (Q[7:4], HEX1);  
    hex_decoder hex_decoder1 (Q[3:0], HEX0);  
  
    always @(posedge clk, negedge aclr)  
        if (!aclr)  
            Q <= 0;  
        else  
            if(Q == 16'b1111111111111111)  
                Q <= 0;  
            else  
                Q <= Q + 1;  
endmodule
```

Główny moduł.

```

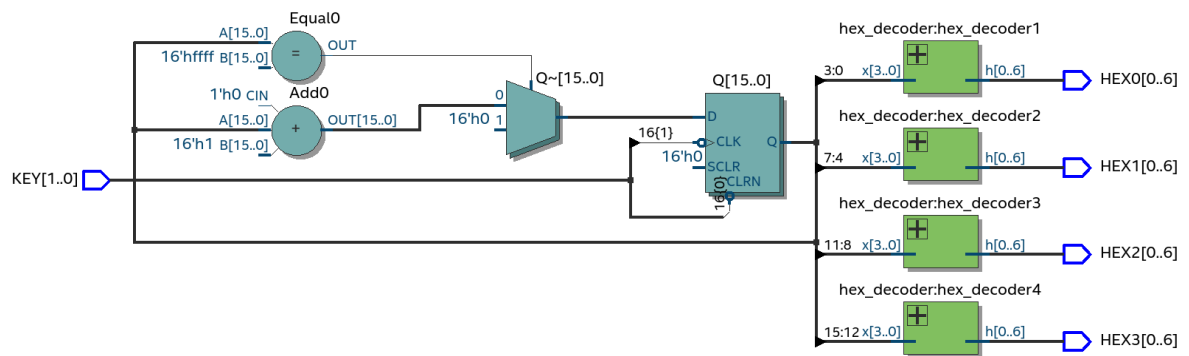
module hex_decoder (
    input [3:0] x,
    output reg [0:6] h);

    always @(*)
    case(x)
        4'b0000: h = 7'b0000001; // 0
        4'b0001: h = 7'b1001111; // 1
        4'b0010: h = 7'b0010010; // 2
        4'b0011: h = 7'b0000110; // 3
        4'b0100: h = 7'b1001100; // 4
        4'b0101: h = 7'b0100100; // 5
        4'b0110: h = 7'b0100000; // 6
        4'b0111: h = 7'b0001111; // 7
        4'b1000: h = 7'b0000000; // 8
        4'b1001: h = 7'b0000100; // 9
        4'b1010: h = 7'b0001000; // A
        4'b1011: h = 7'b1100000; // b
        4'b1100: h = 7'b0110001; // c
        4'b1101: h = 7'b1000010; // d
        4'b1110: h = 7'b0110000; // E
        4'b1111: h = 7'b0111000; // F
    endcase
endmodule

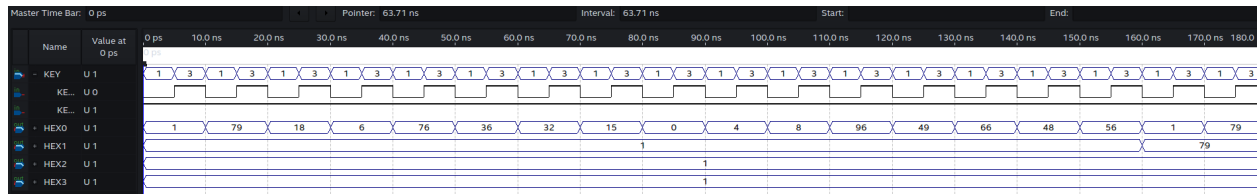
```

Moduł dekodera szesnastkowego.

Wyniki syntezy:



Wyniki symulacji:



3. Zadanie 3

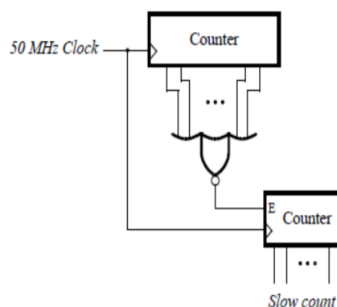
Utwórz moduł licznika modulo `counter_mod_M`, który kolejno wyświetla liczby od 0 do 9 na wyświetlaczu HEX0 z interwałem 1 s.

Wskazówka: skorzystaj z diagramu na Rys. 1, aby zmniejszyć częstotliwość synchronizacji licznika. Aby to zrobić, utwórz projekt licznika `counter_N_bits` na bitach N, a następnie opracuj projekt licznika według modułu `counter_mod_M`, który zlicza do wartości M. (Przy ustalaniu liczby bitów licznika według modułu, użyj funkcji języka Verilog).

```
localparam N=clogb2(M-1);
function integer clogb2(input [31:0] v);
    for (clogb2 = 0; v > 0; clogb2 = clogb2 + 1)
        v = v >> 1;
endfunction
```

Ustal wartość liczby M, aby licznik liczb od 0 do 9 działał w odstępie 1 sekundy.

zby M, aby licznik liczb od 0 do 9 działał w odstępie 1 sekundy.



Rys. 1. Tworzenie wolnego licznika

Kod w języku Verilog:

```
module counter_mod_M
    #(parameter M=8)
(
    input clk, aclr, enable,
    output reg [N-1:0] Q);

    localparam N=clogb2(M-1);
    function integer clogb2(input [31:0] v);
        for (clogb2 = 0; v > 0; clogb2 = clogb2 + 1)
            v = v >> 1;
    endfunction

    always @(posedge clk, negedge aclr)
        if (!aclr) Q <= {N{1'b0}};
        else if(Q == M - 1) Q <= {N{1'b0}};
        else if(enable) Q <= Q + 1'b1;
        else Q <= Q;

endmodule
```

Moduł counter_mod_M

```
]module delay_1_second(
    input clk,
    input areset,
    input enable,
    output [0:6] h);

    localparam sizeA=clogb2(50000000);

    function integer clogb2(input [31:0] v);
        for (clogb2 = 0; v > 0; clogb2 = clogb2 + 1)
            v = v >> 1;
    endfunction

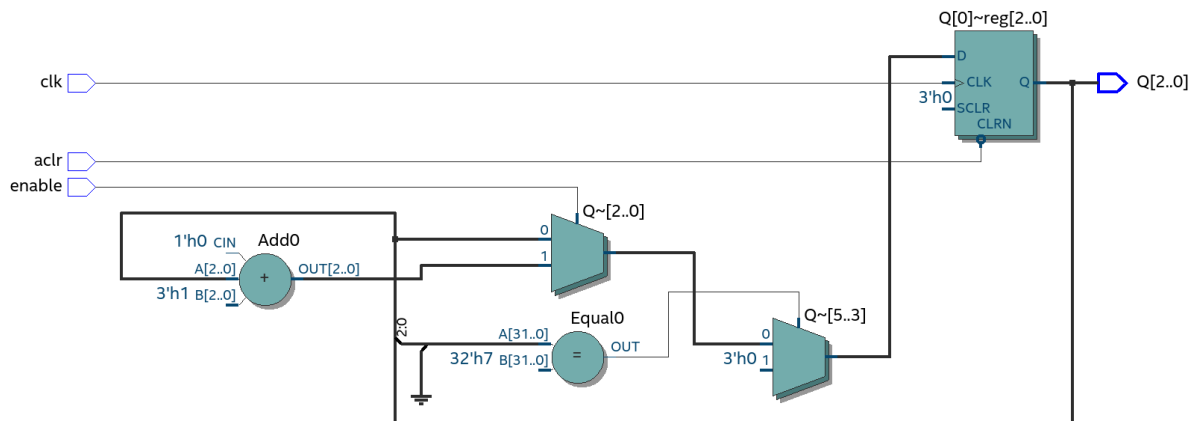
    wire [sizeA-1:0] A;
    wire e;
    wire [3:0] B;

    counter_mod_M#(50000000) ext0(clk, areset, enable, A);
    assign e = ~|A;
    counter_mod_M#(9) ext1 (clk, areset, e, B);
    hex_decoder hd(B, h);

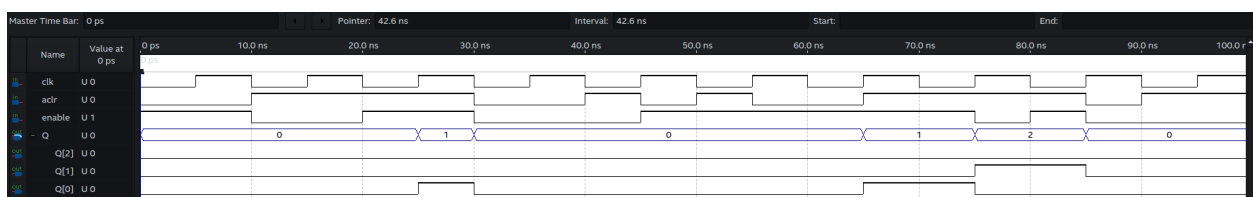
endmodule
```

Moduł nadrzędny wobec counter_mod_M

Wyniki syntezy:



Wyniki symulacji:



4. Zadanie 4

Wyświetl słowo ('FILP') na czterech 7-segmentowych wyświetlaczach. Kody wyświetlanych symboli przekształć za pomocą modułu decoder_7_seg. Obróć słowo na 4 wyświetlaczach w odstępach 1 sekundy w lewo lub w prawo.

Kod (c[1:0])	Symbol (h[0:6])
00	F
01	I
10	L
11	P

Kod w języku Verilog:

```
module switch_between_words(  
    input CLOCK_50,  
    output [0:6] HEX0, HEX1, HEX2, HEX3);  
  
    reg[1:0] position;  
  
    wire clk;  
  
    display_word show_dE1(position[1:0], HEX0, HEX1, HEX2, HEX3);  
    delay_1sec counter_1s(CLOCK_50, clk);  
  
    always @(posedge clk)  
    begin  
        if (position == 2'b00)  
            position <= 3;  
        else  
            position <= position - 1;  
        end  
    end  
endmodule
```

Główny moduł wyświetlający wyraz `FILP`, zliczający czas do przesunięcia oraz przesuwający ciąg liter w lewą stronę.

```
module switch_between_words(  
    input CLOCK_50,  
    output [0:6] HEX0, HEX1, HEX2, HEX3);  
  
    reg [1:0] position;  
  
    wire clk;  
  
    display_word show_dE1(position[1:0], HEX0, HEX1, HEX2, HEX3);  
    delay_1sec counter_1s(CLOCK_50, clk);  
  
    always @(posedge clk)  
    begin  
        if (position == 2'b11)  
            position <= 0;  
        else  
            position <= position + 1;  
        end  
    end  
endmodule
```

Główny moduł wyświetlający wyraz `FILP`, zliczający czas do przesunięcia oraz przesuwający ciąg liter w prawą stronę.

```

module display_word (
    input [1:0] position,
    output [0:6] HEX0, HEX1, HEX2, HEX3);

    reg [1:0] letter_one = 2'b00;
    reg [1:0] letter_two = 2'b01;
    reg [1:0] letter_three = 2'b10;
    reg [1:0] letter_four = 2'b11;

    show_one_letter show1(letter_one[1:0], HEX3);
    show_one_letter show2(letter_two[1:0], HEX2);
    show_one_letter show3(letter_three[1:0], HEX1);
    show_one_letter show4(letter_four[1:0], HEX0);

    always @(*)
    begin
        case(position[1:0])
            2'b00:
                begin
                    letter_one <= 2'b00;
                    letter_two <= 2'b01;
                    letter_three <= 2'b10;
                    letter_four <= 2'b11;
                end
            2'b01:
                begin
                    letter_one <= 2'b01;
                    letter_two <= 2'b10;
                    letter_three <= 2'b11;
                    letter_four <= 2'b00;
                end
            2'b10:
                begin
                    letter_one <= 2'b10;
                    letter_two <= 2'b11;
                    letter_three <= 2'b00;
                    letter_four <= 2'b01;
                end
            2'b11:
                begin
                    letter_one <= 2'b11;
                    letter_two <= 2'b00;
                    letter_three <= 2'b01;
                    letter_four <= 2'b10;
                end
        endcase
    end
endmodule

```

Moduł wyświetlający wyraz na 4 wyświetlaczach w zależności od otrzymanej na wejściu pozycji.

```

module show_one_letter (
    input [1:0] SW,
    output reg [0:6] HEX0;

    always @(*)
    case (SW)
        2'b00: HEX0 = 7'b1110001; //F
        2'b01: HEX0 = 7'b0000110; //I
        2'b10: HEX0 = 7'b0111000; //L
        2'b11: HEX0 = 7'b1110011; //P
    endcase
endmodule

```

Moduł wyświetlający na jednym wyświetlaczu konkretny znak.

```

module delay_1seq(
    input clk,
    output reg out);

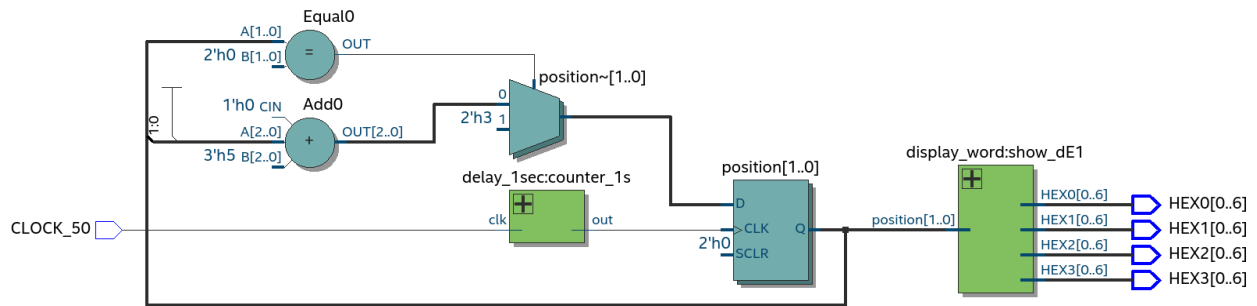
    reg [6:0] count;

    always @(posedge clk)
        if(count == 50)
            begin
                count <= 0;
                out <= 1;
            end
        else
            begin
                count <= count + 1;
                out <= 0;
            end
    end
endmodule

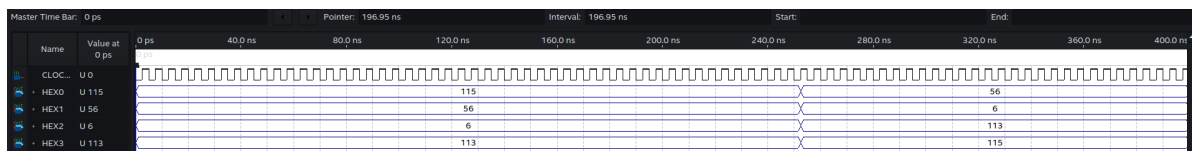
```

Moduł odpowiedzialny za wywołanie opóźnienia do wykonania kolejnego przeskoku. W celu umożliwienia zaprezentowania symulacji w programie Quartus czas ten został zmniejszony do 100 ns.

Wyniki syntezy:



Wyniki symulacji:



5. Wnioski

Zadania zostały wykonane bez problemów.